Series Sync: Enterprise-Architektur & Entwickler-Handoff Blueprint

1. Executive Summary und Zielsetzung

Dieses Dokument konsolidiert den initialen Konzept-Blueprint der Anwendung „Series Sync“ (Automatische Synchronisation von Serien und Filmen) und erweitert diesen um eine tiefgreifende, umsetzbare Architekturspezifikation, die den architektonischen Reifegrad und die Sicherheitsstandards des NexusSaaS-Blueprints adaptiert. Der vorliegende Blueprint definiert eine produktionsreife End-to-End-KI-Pipeline für das vollautomatische, videobasierte Dubbing. Die Architektur spezifiziert Aspekte wie zustandsbehaftete Workflow-Orchestrierung für langlaufende GPU-Prozesse, isoliertes GPU-Sandboxing zum Schutz vor kompromittierten Mediendateien, kryptografische KI-Provenienz-Markierungen gemäß dem EU AI Act, strenge Mandantentrennung auf Datenbankebene sowie hochpräzise Metriken für die audiovisuelle Synchronisation.
Das Resultat ist eine belastbare technische Spezifikation für den Entwickler-Handoff eines Enterprise-ready Minimum Viable Products (MVP). Hierbei wird ausdrücklich auf veraltete oder unsichere Paradigmen verzichtet. So wird beispielsweise der OAuth2 Password Flow gemäß der RFC 9700 Best Current Practice zugunsten von Authorization Code mit PKCE oder etablierten OIDC-Providern gestrichen. Ebenso werden unzulängliche Isolationskonzepte wie reine Docker-Container ohne Kernel-Trennung durch spezialisierte Userspace-Kernel ersetzt, um der Bedrohungslage durch untrusted Code und potenziell bösartige User-Uploads adäquat zu begegnen. Die Plattform richtet sich an Streaming-Plattformen, Studios und Content-Distributoren, die eine hochskalierbare, sichere und rechtlich konforme Lösung zur Lokalisierung von Medieninhalten benötigen.

2. Architektur-Entscheidungsaufzeichnungen (ADRs)

Die folgenden Architektur-Entscheidungsaufzeichnungen (Architecture Decision Records) definieren die technologischen Fundamente der Plattform und grenzen diese von unzureichenden Standardlösungen ab. Sie bilden das Rückgrat der Systemstabilität und Sicherheit.

ID	Entscheidung	Begründung und architektonische Konsequenz
ADR-01	Verwendung von Temporal.io als Orchestrator anstelle von Celery für die Media-Pipeline.	Video-Dubbing ist ein zustandsbehafteter, langlaufender Prozess mit teuren, nicht-idempotenten GPU-Aufrufen. Celery verwirft bei einem Worker-Absturz den gesamten Fortschritt. Temporal bietet hingegen „Durable Execution“ durch Event-Histories, sodass

ID	Entscheidung	Begründung und architektonische Konsequenz
		unterbrochene Videoberechnungen exakt am Fehlerpunkt fortgesetzt werden.
ADR-02	GPU-Workload-Isolierung durch gVisor (nvproxy) anstelle von Firecracker oder Standard-Containern.	Die Ausführung untrusted KI-Modelle erfordert strenge Kernel-Isolierung. Da Firecracker als MicroVM keinen PCIe-Passthrough für GPUs unterstützt, bietet gVisors Userspace-Kernel über das nvproxy-Subsystem den einzig sicheren, multi-tenant-fähigen GPU-Zugriff.
ADR-03	Implementierung des C2PA-Standards als kryptografische Signatur in allen Ausgabe-Artefakten.	Zur Erfüllung von Artikel 50 des EU AI Acts (maschinenlesbare Markierung synthetischer Inhalte) zwingend erforderlich. Die Signatur wird via Python-SDK als JUMBF-Manifest direkt nach der Generierung in die Datei injiziert, um die Anbieterpflichten zu erfüllen.
ADR-04	Mandantenisolation via PostgreSQL Row Level Security (RLS) unter Nutzung von SET LOCAL.	RLS erzwingt die strikte Datentrennung auf Datenbankebene. Die Nutzung von SET LOCAL verhindert ein Leaking des Tenant-Kontexts beim Connection-Pooling, da die Variable am Ende der Transaktion automatisch verworfen wird.
ADR-05	Strikte Sandboxing-Pflicht für jeden FFmpeg-Demultiplexing-Schritt bei User-Uploads.	Sicherheitslücken wie der PixelSmash-Exploit (CVE-2026-8461) ermöglichen Remote Code Execution durch manipulierte Videos. Uploads dürfen ausschließlich in einer gVisor- oder Landlock-isolierten Umgebung entpackt und geparst werden.
ADR-06	Kombination von Wav2Lip mit CodeFormer zur Lippen-Synchronisation und Gesichtsrekonstruktion.	Wav2Lip bietet die beste Lippen-Audio-Synchronisation, degradiert jedoch die visuelle Qualität. CodeFormer wird

ID	Entscheidung	Begründung und architektonische Konsequenz
		GFPGAN vorgezogen, da es eine stabilere zeitliche Konsistenz aufweist und das Flackern (Flickering) zwischen Frames minimiert.
ADR-07	Audio-Time-Stretching mittels WSOLA-Algorithmus anstelle simpler Resampling-Methoden.	Die Anpassung der synthetisierten Audiospur an die originale Videolänge muss ohne Pitch-Shifting (Tonhöhenverzerrung) erfolgen. WSOLA bewahrt die Tonhöhe durch überlappende Waveform-Similarity-Verfahren.
ADR-08	Keine Gleichsetzung von DSGVO Art. 17 (Recht auf Löschung) mit applikationsseitigem Soft-Delete.	Soft-Delete ist lediglich ein Zwischenstatus. Eine echte Löschung erfordert die systematische Bereinigung aktiver Datenbanken, Object Storages, Caches und die Berücksichtigung in Backup-Restore-Prozessen (Deletion Registry).

3. Zielarchitektur und Multi-Mandanten-Datenisolation

Die Systemarchitektur ist modular und asynchron aufgebaut, um der rechenintensiven Natur generativer Video-KI gerecht zu werden. Die Eintrittspforte bildet ein FastAPI-basiertes Gateway, welches Authentifizierung, Autorisierung, Request-Validierung und Rate-Limiting abwickelt. Die Steuerung der komplexen, teils stundenlangen Render-Prozesse übernimmt der Temporal-Orchestrator. Die tatsächliche Verarbeitung findet in einem hochskalierbaren Worker-Pool statt, wobei CPU-intensive Aufgaben (wie Transkription und Demultiplexing) und GPU-intensive Aufgaben (wie Voice-Cloning und Lip-Sync) auf getrennte Knoten-Gruppen geroutet werden. Jeglicher untrusted Code und sämtliche externe Mediendateien werden innerhalb einer dedizierten Sandbox verarbeitet. Die Datenhaltung stützt sich auf PostgreSQL für Geschäfts- und Metadaten, Redis für kurzlebige Token-Buckets und einen Object Storage für die Ablage großer Artefakte, von denen lediglich Referenzen und Hashes im State gespeichert werden.

Die Mandantentrennung in der zentralen PostgreSQL-Datenbank erfolgt zwingend über Row Level Security (RLS) als Defense-in-Depth-Mechanismus. Dieser Ansatz schützt vor Broken Object Level Authorization (BOLA), da ein Durchsickern von Daten bei Applikationsfehlern auf Datenbankebene verhindert wird. Jeder Mandant ist eine First-Class-Entity. Jede Tabelle, die Geschäftsdaten oder Metadaten hält, besitzt eine Spalte für die Tenant-ID und wird durch ALTER TABLE ... FORCE ROW LEVEL SECURITY geschützt.

Die Authentifizierungs-Middleware extrahiert die Tenant-UUID aus dem signierten Token. Bei der Kommunikation mit der Datenbank nutzt das Backend asynchrones Connection Pooling (z.B. via asyncpg). Es ist architektonisch kritisch, den Tenant-Kontext innerhalb des Pools

korrekt zu setzen. Eine Modifikation über SET SESSION würde zum Leaking von Tenant-IDs in recycelten Verbindungen führen, wodurch ein Request fälschlicherweise die Rechte des vorherigen Requests erben könnte. Daher muss zwingend SET LOCAL "app.tenant_id" = '<uuid>' angewendet werden. Dieser Befehl limitiert die Gültigkeit der Variable exakt auf die Lebensdauer der aktuellen Datenbanktransaktion, sodass die Verbindung den Kontext nach dem Commit oder Rollback automatisch verwirft. Die PostgreSQL-Rolle der Applikation darf zudem niemals Tabellen-Owner sein und darf nicht über das Privileg BYPASSRLS verfügen.

4. Zustandsbehaftete Orchestrierung langlaufender KI-Pipelines

Die Orchestrierung von KI-Video-Pipelines erfordert Paradigmen, die über traditionelle Task-Queues hinausgehen. Die architektonische Entscheidung für Temporal anstelle von Celery resultiert direkt aus den Schwächen Letzterer bei zustandsbehafteten, mehrstufigen Prozessen. Celery ist für "Fire-and-Forget"-Szenarien konzipiert, bietet jedoch keine native Event-History oder robustes Checkpointing für komplexe Graphen. Ein Absturz eines Celery-Workers während eines langwierigen GPU-Renderings erfordert oftmals den Neustart des gesamten Jobs, was erhebliche Infrastrukturkosten nach sich zieht und die Idempotenz gefährdet. Ebenso ist LangGraph primär auf reaktive LLM-Agenten-Abläufe optimiert, bietet jedoch nicht die gleiche operative Ausfallsicherheit für netzwerkintensive, langlaufende Medientransformationen wie Temporal.
Temporal begegnet diesen Herausforderungen durch das Konzept der "Durable Execution". Der Code des Workflows wird als gewöhnliche Funktion geschrieben, muss jedoch zwingend deterministisch sein. Das bedeutet, dass direkte Netzwerkaufrufe, Zufallszahlen oder zeitabhängige Funktionen (wie datetime.now()) innerhalb der Workflow-Funktion untersagt sind. Alle nebenläufigen Effekte werden in sogenannte Activities ausgelagert. Der Temporal-Server zeichnet den Beginn, das Ende und das Ergebnis jeder Activity in einem Append-Only-Log (Event-History) auf. Stürzt der ausführende Worker ab, wird der Workflow auf einem anderen Knoten neu gestartet. Dabei spielt Temporal die Event-Historie ab: Bereits abgeschlossene Activities werden nicht erneut ausgeführt; die Engine liefert stattdessen direkt die persistierten Rückgabewerte und setzt die Ausführung exakt am Punkt des Absturzes fort.
Für die Verarbeitung ganzer Serienepisoden wird der Media-Stream in kleinere Chunks unterteilt. Jede Activity sendet während der Verarbeitung regelmäßige Heartbeats an den Temporal-Server, in denen der Fortschritt (beispielsweise die ID des zuletzt verarbeiteten Chunks) übermittelt wird. Bei einem Infrastrukturausfall nutzt der neue Worker diese Heartbeat-Details, um die Berechnung genau beim letzten fehlgeschlagenen Chunk aufzunehmen, anstatt das gesamte Video erneut zu rendern. Dieses Modell garantiert, dass transiente Fehler, wie ein Timeout bei einer externen LLM-API, isoliert auf Activity-Ebene mit exponentiellem Backoff wiederholt werden. Dauerhafte Fehler, wie korrupte Input-Dateien oder Budget-Erschöpfung, werden hingegen durch eine granulare Retry-Policy sofort als nicht-wiederholbar an die Workflow-Ebene gemeldet. Dies verhindert Endlosschleifen und unkontrollierte Cloud-Kosten.

5. GPU-Sandboxing und Infrastruktur-Sicherheit

Da die Generierungsprozesse die Ausführung von untrusted Machine-Learning-Modellen und

das Parsen nutzergenerierter Videodateien bedingen, ist die Sicherheit auf Host-Ebene von höchster Priorität. Konventionelle Docker-Container teilen sich den Host-Kernel und bieten Angreifern bei Kernel-Vulnerabilitäten einen direkten Weg zur Host-Übernahme. Dies ist besonders kritisch bei der initialen Medien-Ingestion über FFmpeg. FFmpeg unterstützt hunderte von komplexen Media-Formaten und ist historisch fehleranfällig. Jüngste Schwachstellen wie der PixelSmash-Exploit (CVE-2026-8461) oder CVE-2026-18393 belegen dies eindrucksvoll. Beim PixelSmash-Exploit führt ein Out-of-Bounds-Write im MagicYUV-Decoder zu einer Speicherüberlauf-Attacke. Diese ermöglicht es einem Angreifer, durch das bloße Hochladen einer präparierten AVI- oder MKV-Datei, die serverseitig (etwa zur Thumbnail-Generierung) geparst wird, Denial-of-Service-Zustände auszulösen oder beliebigen Code mit den Rechten des FFmpeg-Prozesses auszuführen. Der Angriff erfordert keine Authentifizierung und erfolgt vollständig geräuschlos.
Um diese Bedrohungen zu neutralisieren, dürfen Medien-Uploads ausschließlich in einer isolierten Umgebung verarbeitet werden. Während MicroVMs wie Firecracker eine exzellente CPU-Isolation bieten, schließen sie PCIe-Device-Passthrough für GPUs aus Designgründen aus, um die Angriffsfläche zu minimieren. Somit sind Firecracker-Umgebungen standardmäßig CPU-only und für GPU-beschleunigtes Deep Learning ungeeignet. Die Architektur verlässt sich daher auf gVisor als hochsicheren Userspace-Kernel. gVisor fängt alle System Calls der Anwendung ab, führt sie im Userspace aus und leitet nur eine stark gefilterte Untermenge an den Host-Kernel weiter.
Für den GPU-Support aktiviert gVisor das Subsystem nvproxy. Dieses Proxy-Modul reicht Aufrufe an den Host-NVIDIA-Treiber gezielt weiter (beispielsweise an /dev/nvidiactl und /dev/nvidia-uvm), nachdem alle ioctl-Befehle gegen eine restriktive, interne Whitelist validiert wurden. Die Konfiguration erfolgt über die Container Device Interface (CDI) Spezifikation. Anstatt fehleranfällige Prestart-Hooks des alten NVIDIA-Container-Toolkits zu nutzen, wird runsc (die gVisor-Laufzeitumgebung) direkt als Low-Level-Runtime via Containerd aufgerufen. Containerd wendet die CDI-Spezifikationen an, was Symlinks und Treiber-Mounts sicher in den isolierten Userspace des gVisor-Sentrys mappt. Dieser Ansatz ermöglicht sicheres Slicing der GPU-VRAMs, blockiert Treiber-basierte CVEs und stellt sicher, dass selbst bei einem Ausbruch aus dem FFmpeg-Subprozess der Host-Kernel sowie Applikations-Secrets unangetastet bleiben.

6. Medien-Ingestion, Härtung und Audio-Stem-Separation

Nachdem das hochgeladene Video in der gesicherten gVisor-Sandbox empfangen wurde, beginnt die Extraktion und Separierung der Audiospuren. Um ein professionelles Dubbing zu erzeugen, muss die Originalstimme des Schauspielers entfernt werden, während Hintergrundmusik, Umgebungsgeräusche und Soundeffekte erhalten bleiben. Dieser Schritt wird durch HTDemucs (Demucs v4) realisiert, einem State-of-the-Art-Modell für Music Source Separation aus der Meta AI Forschung.
HTDemucs v4 übertrifft ältere Modelle wie Spleeter durch eine hybride Architektur. Es kombiniert ein U-Net, welches direkt auf der zeitlichen Audio-Wellenform (Waveform) operiert, mit einem parallel laufenden U-Net, das Frequenz-Spektrogramme (STFT) analysiert. Diese beiden Analysepfade werden im Kern durch einen Cross-Domain-Transformer verbunden. Diese Architektur erlaubt es dem Modell, zeitliche Präzision (wie die Schärfe von perkussiven Anschlägen) mit spektraler Genauigkeit zu vereinen, wodurch Signal-to-Distortion-Ratios (SDR)

von über 9.0 dB auf Hochqualitäts-Benchmarks erreicht werden.
In der Pipeline wird HTDemucs über den Parameter --two-stems=vocals konfiguriert. Dies forciert das Modell, das Audio-File effizient in genau zwei Spuren zu trennen: Eine isolierte Vocal-Spur, die an die nachfolgenden Transkriptions-Module weitergereicht wird, und eine Begleitspur (Accompaniment), die alle anderen Instrumente und Geräusche enthält. Da HTDemucs ressourcenintensiv ist und auf reinen CPUs zu Out-of-Memory-Abbrüchen oder inakzeptablen Laufzeiten neigen kann, wird die Ausführung zwingend auf GPU-gestützte Worker-Knoten geroutet, wo das Modell große Audio-Batches in Bruchteilen der Echtzeit verarbeiten kann.

7. Sprechererkennung (Diarization) und Word-Level-Alignment

Die isolierte Sprachspur muss nun in Text überführt und den jeweiligen Sprechern zugeordnet werden. Die Pipeline kombiniert hierfür spezialisierte Modelle, um die Limitierungen standardisierter ASR-Systeme (Automatic Speech Recognition) zu überwinden.

7.1 Speaker Diarization mit Pyannote 3.1

Um festzustellen, welche Person zu welchem Zeitpunkt spricht (Speaker Diarization), setzt die Architektur auf das pyannote.audio 3.1 Framework. Pyannote verarbeitet das Audiosignal in überlappenden Sliding-Windows (typischerweise in 10-Sekunden-Fenstern mit starker Überlappung). In einem ersten Schritt segmentiert ein neuronales Netz (Segmentation Model) die Regionen in Sprache und Stille und weist jedem Frame Wahrscheinlichkeitsvektoren zu, um überlappende Sprache (Overlapped Speech Detection) von mehreren gleichzeitigen Sprechern aufzulösen.
Anschließend wird für jeden detektierten Sprachabschnitt ein hochdimensionaler Feature-Vektor (Embedding) durch ein ResNet-basiertes Modell extrahiert. Diese Embeddings werden mittels Agglomerative Hierarchical Clustering (AHC) basierend auf Kosinus-Ähnlichkeit gruppiert. So wird sichergestellt, dass derselbe Sprecher auch über verschiedene Szenen hinweg konsistent mit einer eindeutigen Sprecher-ID versehen wird.

7.2 Forced Alignment mit WhisperX

Für die eigentliche Transkription kommt WhisperX zum Einsatz. Das Basis-Whisper-Modell von OpenAI liefert zwar exzellente Textgenauigkeit, versagt jedoch bei der exakten zeitlichen Verortung der Wörter. Die Zeitstempel von Standard-Whisper basieren auf Segment-Schätzungen, die stark driften und oftmals Abweichungen von mehreren Sekunden aufweisen (±500 ms und mehr). Für visuelles Lip-Sync ist jedoch eine Präzision auf Frame-Ebene zwingend erforderlich.
WhisperX löst dieses Problem durch eine mehrstufige Pipeline. Zunächst filtert ein Silero VAD-Modell absolute Stille heraus, was eine stark beschleunigte, gebatchte Transkription auf der GPU erlaubt (bis zu 70-fache Echtzeit). Nach der Rohtranskription greift das Forced-Alignment-Modul. WhisperX nutzt hierfür ein phonembasiertes wav2vec2-Modell. Der Aligner kennt bereits den von Whisper generierten Text; seine einzige Aufgabe ist es, die vorhergesagte Phonem-Sequenz mit der tatsächlichen akustischen Wellenform abzugleichen. Durch die Suche nach dem optimalen Pfad durch die Audio-Frames generiert WhisperX

Wort-Zeitstempel mit einer extremen Präzision von ±50 Millisekunden.
Diese Methode ist dem traditionellen Montreal Forced Aligner (MFA) überlegen. Der MFA basiert auf einer älteren GMM-HMM-Architektur (Gaussian Mixture Models / Hidden Markov Models) und erfordert aufwendig kuratierte Aussprache-Wörterbücher (Pronunciation Dictionaries) sowie Grapheme-to-Phoneme (G2P) Modelle zur Behandlung unbekannter Wörter (Out-Of-Vocabulary). Zwar bietet der MFA in sauberen Laborbedingungen hervorragende Genauigkeit, ist jedoch in unstrukturierten, realen Video-Szenarien (In-the-Wild) mit Hintergrundrauschen weniger robust und erfordert signifikant mehr manuellen Konfigurationsaufwand. Die Entscheidung fällt daher klar auf WhisperX für das automatische Alignment.

8. Sprachsynthese und Audio-Time-Stretching ohne Artefakte

Die generierten, sprecherseparierten Transkripte werden unter Berücksichtigung von Slang und kulturellem Kontext durch LLMs in die Zielsprache übersetzt. Ein Voice-Cloning-Modell generiert anschließend für jede Sprecher-ID (aus Pyannote) eine zielgerichtete Synthese. Ein fundamentales Problem beim Dubbing ist jedoch die abweichende Satzlänge der Zielsprache. Wenn ein deutscher Satz länger oder kürzer ist als das englische Original, muss das synthetisierte Audio in das vorgegebene Zeitfenster der Szene eingepasst werden.
Ein simples Beschleunigen oder Verlangsamen des Audios durch Resampling verändert die Frequenz des Signals und führt zu unerwünschten Pitch-Shifts (dem sogenannten Chipmunk-Effekt). Um die Tonhöhe bei der Zeitstreckung absolut konstant zu halten, integriert die Plattform den WSOLA-Algorithmus (Waveform Similarity Overlap-Add) über die Rubberband-Bibliothek in FFmpeg.
Anstatt die Samples selbst zu strecken, zerteilt WSOLA das Audiosignal in kleine, überlappende Frames. Soll das Audio verlangsamt werden, werden diese Frames weiter auseinandergezogen; soll es beschleunigt werden, rücken sie näher zusammen. Um die unweigerlich entstehenden Diskontinuitäten an den Schnittstellen zu verbergen, nutzt der Algorithmus Cross-Correlation. Er sucht innerhalb eines Toleranzfensters nach dem exakten Punkt, an dem die Phasen der Wellenformen optimal übereinstimmen (Waveform Similarity). Anschließend werden die Frames über eine Hann-Fenster-Kreuzblende nahtlos zusammengefügt. WSOLA garantiert somit eine exzellente Audioqualität für vokale Signale, ohne die Identität des Stimmklons zu verzerren.

9. Lip-Sync-Synthese und Gesichtswiederherstellung

Das Herzstück der visuellen Adaptation ist die Manipulation der Mundpartie im Originalvideo. Modelle wie MuseTalk, LatentSync oder Wav2Lip synthetisieren auf Basis der neuen Audiospur frame-genau Lippenbewegungen.
Die Architektur evaluiert kontinuierlich Modelle, definiert jedoch als Baseline für das MVP eine Kombination aus Wav2Lip und einem Face-Restoration-Netzwerk. Wav2Lip nutzt ein Generative Adversarial Network (GAN), das gegen einen eingefrorenen Lip-Sync-Experten (SyncNet) trainiert wurde. Dies zwingt den Generator, hochakkurate Lippenbewegungen zu erzeugen, die selbst auf ungesehenen Gesichtern und in extremen Winkeln funktionieren. Der gravierende Nachteil von Wav2Lip ist jedoch die visuelle Degradation: Die generierte

Mundpartie wirkt oft unscharf, verliert Hauttexturen und fügt sich inkonsistent in das restliche, hochauflösende Gesicht ein.
Um diese Artefakte zu eliminieren, durchläuft jeder veränderte Frame einen zwingenden Post-Processing-Schritt zur Gesichtswiederherstellung (Face Restoration). Die Architektur favorisiert hierbei CodeFormer gegenüber GFPGAN. Zwar liefern beide Algorithmen durch die Injektion generativer Prior-Informationen hervorragende Schärfe, jedoch zeigt CodeFormer eine überlegene zeitliche Konsistenz. Es bewahrt die Gesichtsidentität der Schauspieler besser und minimiert das unnatürliche Flackern (Flickering) zwischen aufeinanderfolgenden Frames, das bei GFPGAN in Videosequenzen häufig auftritt. Der Fidelity-Parameter des Modells wird dynamisch justiert, um den optimalen Kompromiss zwischen Detailreichtum und Identitätserhalt zu erzielen.
Alternativmodelle wie LatentSync, welche auf Stable-Diffusion-Architekturen basieren, bieten zwar intrinsisch schärfere Bilder ohne Post-Processing, erfordern jedoch ein Vielfaches an VRAM und weisen deutlich höhere Inferenzzeiten auf, was sie für extrem lange Videoprozesse kostenintensiver macht.

10. Qualitätssicherung, Metriken und Audio-Mastering

Da die menschliche Wahrnehmung hochsensibel auf asynchrone Lippenbewegungen reagiert, ist eine manuelle Qualitätskontrolle für massenhafte Serienproduktionen ineffizient. Die Pipeline etabliert daher automatisierte, mathematische Metriken als QA-Gates.
Die Evaluierung basiert auf dem SyncNet-Framework, welches visuelle und auditive Embeddings extrahiert und deren zeitliche Ausrichtung vergleicht. Daraus leiten sich zwei Kernmetriken ab:

- **LSE-D (Lip-Sync Error Distance):** Misst den Euklidischen Abstand im hochdimensionalen Embedding-Raum zwischen der generierten Mundbewegung und dem Audiosignal. Niedrigere Werte bedeuten eine höhere Synchronisation.
- **LSE-C (Lip-Sync Error Confidence):** Misst das Vertrauen des Diskriminator-Netzwerks in die Synchronizität der Modalitäten. Höhere Werte indizieren eine robuste, fehlerfreie Synchronisierung.

Jedes gerenderte Video-Segment durchläuft diesen Evaluierungs-Knoten. Überschreitet der LSE-D-Wert einen kritischen Schwellenwert (z.B. > 9.5) oder fällt die Konfidenz LSE-C unter einen Mindestwert (z.B. < 4.5), wird das Artefakt automatisch verworfen. Der Temporal-Orchestrator iteriert dann mit veränderten Modellparametern oder eskaliert das Segment an einen Human-in-the-Loop (HITL) Workflow zur manuellen Überprüfung.
Im finalen Schritt erfolgt das Audio-Mastering. Der isolierte Hintergrund-Stem wird mit den neuen, zeitlich gestreckten Vocals gemischt. Das finale Audio wird nach den Broadcast-Vorgaben von Netflix normalisiert: Die zielgerichtete Dialog-Lautheit (dialogue-gated) wird auf -27 LKFS (±2) gepegelt, und ein True-Peak-Limiter bei -2 dBTP gesetzt. Diese Netflix-Vorgabe unterscheidet sich vom EBU-R128-Standard (der oft bei -23 LUFS für die gesamte Programmlautheit liegt) und stellt sicher, dass die dynamische Bandbreite eines Kinomixes bewahrt bleibt und die Sprachverständlichkeit auf Enterprise-Niveau liegt.

11. Compliance: DSGVO, Biometrie und EU AI Act

Die Verarbeitung audiovisueller Daten, das Klonen von Stimmen und die Generierung synthetischer Videos berühren fundamentale europäische Rechtsnormen, deren technische

Einhaltung tief in die Systemarchitektur integriert sein muss.

11.1 Biometrische Daten und DSGVO-Konformität

Geklonte Stimmen (Voiceprints) klassifizieren als biometrische Daten nach Artikel 9(2) DSGVO. Der Europäische Datenschutzausschuss (EDPB) legt fest, dass die Erhebung und Speicherung derartiger Templates ausschließlich auf Basis einer ausdrücklichen, spezifischen und informierten Einwilligung erfolgen darf. Das System dokumentiert diese Einwilligungen lückenlos.
Besonderes Augenmerk erfordert das Recht auf Löschung (Artikel 17 DSGVO). Ein applikationsseitiges Soft-Delete ist rechtlich unzureichend. Widerruft ein Nutzer die Einwilligung, löst das System eine automatisierte Löschpipeline aus, die Voiceprint-Vektoren unwiderruflich aus aktiven PostgreSQL-Datenbanken, Redis-Caches, S3-Object-Storages und Protokollen entfernt. Für unveränderliche Backups wird ein Deletion Registry gepflegt, das sicherstellt, dass zu löschende Daten im Falle eines Restores automatisiert und vor Wiederinbetriebnahme vernichtet werden.

11.2 EU AI Act und C2PA-Watermarking

Mit dem Inkrafttreten der Transparenzpflichten aus Artikel 50 des EU AI Acts im August 2026 müssen KI-generierte oder manipulierte Video- und Audio-Inhalte zwingend gekennzeichnet werden. Die Verordnung unterscheidet zwischen der sichtbaren Offenlegungspflicht für den Betreiber (Deployer) und der technischen, maschinenlesbaren Markierungspflicht für den Systemanbieter (Provider).
Um die Provider-Pflichten zu erfüllen, implementiert die Plattform den offenen C2PA-Standard (Content Credentials). Unmittelbar nach der finalen Video-Assemblierung integriert ein dedizierter Post-Processing-Worker über das Python-SDK (c2pa-python) ein kryptografisch signiertes Manifest in die Header der Mediendatei.
Dieses Manifest nutzt das JUMBF-Format (JPEG Universal Metadata Box Format) und enthält detaillierte Assertions, darunter die explizite c2pa.ai-disclosure. Sie dokumentiert lückenlos, welche KI-Modelle angewendet wurden und welche Aktionen (z. B. c2pa.edited, c2pa.created) auf das Ursprungsmaterial einwirkten. Die Signatur erfolgt mittels X.509-Zertifikaten, die von anerkannten Certificate Authorities (wie DigiCert) bezogen und in sicheren Hardware Security Modulen (HSMs) verwaltet werden, um die Provenienz manipulationssicher zu belegen. Um dem Problem des Entfernens von Metadaten auf Social-Media-Plattformen zu begegnen, wird das Manifest zusätzlich über Soft-Bindings (wie SynthID Audio/Video Watermarking) direkt in die audiovisuellen Signale eingebettet. Diese imperzeptiblen Wasserzeichen überstehen Videokompressionen und dienen als persistenter Verweis auf das ursprüngliche C2PA-Manifest.

12. Nichtfunktionale Anforderungen (NFRs)

Die folgenden Architektur-Kriterien müssen unter Last erfüllt werden, um die Freigabe des Systems für den Produktionsbetrieb zu rechtfertigen:

Kategorie	MVP-Akzeptanzkriterium (Zielwert)
API-Latenz & Responsiveness	P95 < 500 ms für alle synchronen Aufrufe (Auth, Job-Validierung, Status-Polling). Die asynchrone Jobannahme wird innerhalb von 2

Kategorie	MVP-Akzeptanzkriterium (Zielwert)
	Sekunden bestätigt.
Ausfallsicherheit & Idempotenz	Ein harter Worker-Absturz (SIGKILL) während des GPU-Renderings führt dank Temporal zum deterministischen Resume des Workflows ab dem zuletzt abgeschlossenen Chunk. Externe Stripe-Webhooks lösen keine doppelten Abrechnungen aus.
Ressourcen & Pipeline-Durchsatz	Der GPU-VRAM ist durch CDI/gVisor strikt limitiert; keine Out of Memory-Ereignisse beeinträchtigen benachbarte Container. Die WhisperX-Inferenz skaliert durch Batching auf mindestens 50-fache Echtzeit.
Sandbox-Sicherheit	Der Netzwerk-Egress ist in der gVisor-Sandbox strikt limitiert. Es gibt keinen Zugriff auf den ungeschützten Host-Kernel, keine internen Cloud-Metadaten und keine Erreichbarkeit von Produktions-Secrets aus dem Container heraus.
Data Privacy & Observability	Zwingendes Fehlen sensibler Keys, biometrischer Templates oder Klartext-Personendaten in den Standard-Logs. Die generierten Trace-IDs erlauben ein vollständig pseudo-anonymisiertes Debugging über alle Systemgrenzen hinweg.

13. Teststrategie, Implementierungsphasen und Repository

Die Architektur erlaubt keine Deployment-Freigabe ohne das Bestehen einer kompromisslosen Test-Hierarchie, angefangen bei Unit-Tests (z.B. für Temporal-Retries), über Integrationstests (Validierung von PostgreSQL RLS Policies), bis hin zu Security-Tests (SAST-Pipelines und Sandbox-Boundary-Simulationen) und End-to-End-Prozessen (Upload bis C2PA-Auslieferung). Der Entwickler-Handoff sieht eine gestaffelte Implementierung vor, um operationelle Risiken frühzeitig zu mitigieren:

- **Phase 0 (Foundation):** Validierung der ADRs, Setup des Cloud-Providers mit Support für Nested Virtualization (notwendig für gVisor-Performance) und Initialisierung der C2PA-Zertifikatsstrukturen.
- **Phase 1 (Plattform):** Bereitstellung von FastAPI, PostgreSQL inkl. RLS-Migrationen, Redis für Token-Buckets und Deployment des Temporal-Clusters.
- **Phase 2 (Identity & Auth):** OIDC-Integration, RBAC, Durchsetzung der Tenant-Isolierung.
- **Phase 3 (Orchestrierung):** Definition der deterministischen Temporal-Workflows, Implementierung der Chunking-Logik und der Retry-Policies.
- **Phase 4 (Audio/ASR-Pipeline):** Integration von HTDemucs, Pyannote 3.1 und WhisperX als dedizierte Temporal-Activities innerhalb der gVisor-Sandbox.
- **Phase 5 (Synthesis & Video):** Implementierung des Voice-Clonings, Wav2Lip,

CodeFormer-Restaurierung und des WSOLA-Audio-Mixings über Rubberband.
- **Phase 6 (QA & Compliance):** Einbindung des LSE-D/LSE-C-Evaluators und des c2pa-python-Injectors.
- **Phase 7 (Pilot):** Lasttests, Stripe-Billing-Integration und Validierung der NFRs unter realistischen Auslastungsszenarien.

Repository-Struktur

Das Projekt folgt einer klaren Monorepo-Struktur, die fachliche Domänen strikt von Infrastruktur und Applikations-Routing trennt:
nexussaas-synclab/ ├── apps/ │ ├── api/ # FastAPI Endpunkte, Auth-Middlewares, Webhooks │ ├── orchestrator/ # Temporal Workflow Definitionen (deterministischer Code) │ └── workers/ # Temporal Activities (Audio/Video Inference, C2PA) ├── packages/ │ ├── database/ # PostgreSQL Schemata, Alembic Migrations, RLS Policies │ ├── media_processing/ # FFmpeg Wrapper, HTDemucs (Audio Separation), WSOLA │ ├── ml_pipeline/ # WhisperX, Pyannote, Wav2Lip, CodeFormer Adapter │ ├── security_c2pa/ # c2pa-python SDK Integration, Manifest-Generierung │ └── qa_metrics/ # SyncNet Evaluator (LSE-C, LSE-D Metriken) ├── infrastructure/ │ ├── temporal_cluster/ # IaC für Temporal Server (Frontend, History, Matching) │ └── sandbox_gvisor/ # runsc, nvproxy Konfiguration, CDI Definitionen ├── tests/ │ ├── unit/ │ ├── integration/ │ ├── security/ │ └── e2e/ └── docs/ # ADRs, Runbooks, Threat Models, Art-17-Löschkonzepte
Abgrenzung: Dieser Blueprint ist ein detaillierter Architekturentwurf für Senior Engineers. Vor dem Produktionsbetrieb sind Quellcode, Cloud-Konfigurationen, Penetrationstests, die juristische Prüfung von Voice-Cloning-Einwilligungen sowie ausgiebige Lasttests der GPU-Cluster separat zu prüfen und formell freizugeben.

Quellenangaben

1. AI-generated content: where do you start with AI Act, C2PA and, https://qbankdam.com/en/blog/ai-generated-content-ai-act-c2pa-disclosure 2. AI Lip-Sync Model Selection Guide 2026: MuseTalk, LatentSync, https://tomodahinata.com/en/blog/ai-lip-sync-talking-head-model-selection-guide-2026 3. C2PA Implementation Guidance, https://spec.c2pa.org/specifications/specifications/2.4/guidance/Guidance.html 4. Introduction to gVisor security, https://gvisor.dev/docs/architecture_guide/intro/ 5. Understanding Sandboxes: gVisor, Hypervisors, and Firecracker, https://www.salmanq.com/blog/understanding-sandboxes/ 6. Demucs download | SourceForge.net, https://sourceforge.net/projects/demucs.mirror/ 7. Row Level Security for Tenants in Postgres | Crunchy Data Blog, https://www.crunchydata.com/blog/row-level-security-for-tenants-in-postgres 8. Why PostgreSQL Row-Level Security Is the Right Approach to, https://dev.to/dvoraj75/why-postgresql-row-level-security-is-the-right-approach-to-django-multitenancy-3e1m 9. [question]: supporting row level security (RLS) in Postgres SQL #6137, https://github.com/mikro-orm/mikro-orm/discussions/6137 10. PostgreSQL RLS: your last defense against tenant data leaks, https://mvpfactory.io/blog/postgresql-row-level-security-for-multi-tenant-saas-eliminating-tenant-data/ 11. Postgres Row-Level Security: Policies, Pitfalls, Performance, https://rivestack.io/blog/postgresql-row-level-security 12. Celery vs Temporal: Workflow Engine for AI Jobs - CodeNicely,

https://codenicelystudio.com/blog/businesses/saas/celery-vs-temporal-workflow-engine-ai-jobs 13. Why We Moved Agent Pipelines from Celery to Temporal · ZhuoQi Dev, https://zhuoqidev.com/en/posts/why-temporal-not-celery/ 14. From Celery/Redis to Temporal: A Journey Toward Idempotency, https://dev.to/wintrover/from-celeryredis-to-temporal-a-journey-toward-idempotency-and-reliable-workflows-k1i 15. Why Temporal Is Better Than LangGraph for Long-Running AI, https://www.alongside.team/blog/temporal-vs-langgraph-long-running-ai-workflows 16. Temporal vs LangChain vs LangGraph (2026): When to Use Each, https://cordum.io/blog/temporal-vs-langchain 17. Celery vs Temporal.io Comparison - pedrobuzzi.dev, https://pedrobuzzi.hashnode.dev/celery-vs-temporalio 18. Temporal for Data Workflows: Durable Execution, Retries & Long, https://pipecode.ai/blogs/temporal-data-workflows-durable-execution-retries-long-running-pipelines 19. Render Workflows vs Temporal: managed compute vs your own, https://render.com/articles/render-workflows-vs-temporal 20. Critical FFmpeg Vulnerability: CVE-2026-8461 (PixelSmash), https://elements.tv/news/critical-ffmpeg-vulnerability-cve-2026-8461-pixelsmash/ 21. PixelSmash (CVE-2026-8461): Critical FFmpeg Flaw | JFrog, https://jfrog.com/blog/pixelsmash-critical-ffmpeg-vulnerability-turns-media-files-into-weapons/ 22. CVE-2026-18393: FFmpeg TDSC RCE Vulnerability - SentinelOne, https://www.sentinelone.com/vulnerability-database/cve-2026-18393/ 23. What Are GPU Sandboxes? How Isolation Works When Agents, https://www.thundercompute.com/blog/what-are-gpu-sandboxes 24. GPU sandboxes: isolation models and platform support in 2026 | Blog, https://northflank.com/blog/gpu-sandboxes 25. Firecracker vs gVisor: How Each Isolates Untrusted Code, and Why, https://fly.io/learn/firecracker-vs-gvisor/ 26. GPU Support - gVisor, https://gvisor.dev/docs/user_guide/gpu/ 27. gVisor - nvproxy - HackingNote, https://www.hackingnote.com/en/gvisor/nvproxy/ 28. Running nvproxy containers without Docker #9435 - gvisor - GitHub, https://github.com/google/gvisor/issues/9435 29. [Sandbox] gVisor · Issue #521 · cncf/sandbox - GitHub, https://github.com/cncf/sandbox/issues/521 30. Slicing a GPU Between Untrusted Containers - Midokura, https://midokura.com/slicing-a-gpu-between-untrusted-containers/ 31. Demucs Online Tutorial: How to Separate Stems Without Installing, https://stemsplit.io/blog/demucs-online-tutorial 32. Demucs v4 Complete Guide: Running Meta's Source-Separation, https://tomodahinata.com/en/blog/demucs-v4-music-source-separation-production-guide 33. Demucs vs Spleeter - The Ultimate Guide | BeatsToRapOn Blog, https://beatstorapon.com/blog/demucs-vs-spleeter-the-ultimate-guide/ 34. GitHub - facebookresearch/demucs: Code for the paper Hybrid, https://github.com/facebookresearch/demucs 35. Complete Guide to Setting Up Demucs Locally for AI Stem Separation, https://dev.to/stevecase430/complete-guide-to-setting-up-demucs-locally-for-ai-stem-separation-580h 36. pyannote/pyannote-audio: Neural building blocks for speaker, https://github.com/pyannote/pyannote-audio 37. speaker diarization with adapted pyannote.audio, https://praat.org/manual/speaker_diarization_with_adapted_pyannote_audio.html 38. Pyannote 3.1 Diarization Pipeline - Emergent Mind, https://www.emergentmind.com/topics/pyannote-3-1-based-pipeline 39. pyannote/overlapped-speech-detection - Hugging Face, https://huggingface.co/pyannote/overlapped-speech-detection 40. pyannote-audio/README.md

· main - GitLab,
https://gitlab.cognitive-ml.fr/ralgayres/wav2boundaries/-/blob/main/pyannote-audio/README.md 41. What Is WhisperX? Word Timestamps, Diarization, Cost (2026),
https://www.forasoft.com/learn/ai-for-video-engineering/articles-ai/whisperx-diarization-word-level-timestamps 42. WhisperX with Diarization | Guides - Clore.ai,
https://docs.clore.ai/guides/audio-and-voice/whisperx 43. WhisperX and forced alignment, explained - Whipscribe, https://whipscribe.com/blog/whisperx-forced-alignment-guide-2026 44. Learn with me: What is Montreal Force Alignment open source tool,
https://stanislav-kirdey.medium.com/learn-with-me-what-is-montreal-force-alignment-open-source-tool-27fe7bd7179b 45. Montreal Forced Aligner Overview - Emergent Mind,
https://www.emergentmind.com/topics/montreal-forced-aligner 46. Montreal Forced Aligner Tutorial - Miao ZHANG, https://miaozhang.org/2025-04-28-montreal-forced-aligner-tutorial/ 47. Montreal Forced Aligner and the state of speech-to-text ... - arXiv,
https://arxiv.org/html/2606.18466v1 48. English alignment benchmarks — mfa model 3.1.2 documentation,
https://mfa-models.readthedocs.io/en/latest/benchmarks/english_alignments.html 49. Beyond Lip Sync:Reference-Grounded Oral Refinementfor Audio, https://arxiv.org/html/2609.38019v1 50. Lip Sync Translation: Match the Lips to the Translated Speech,
https://www.technolynx.com/post/matching-lips-to-translated-speech-not-the-other-way/ 51. WSOLA Explained: How Audio Gets Slowed Down Without,
https://www.practicesession.app/blog/wsola-deep-dive/ 52. Rubber Band Audio Time Stretcher Library - Breakfast Quay, https://breakfastquay.com/rubberband/integration.html 53. Time-Scale Modification Algorithms for Music Audio Signals,
https://audiolabs-erlangen.de/content/05_fau/professor/00_mueller/01_group/2011_DriedgerJonathan_TSM_MasterThesis.pdf 54. Audio Time Stretching - Implementation in Pure Java Using WSOLA,
https://0110.be/posts/Audio_Time_Stretching_-_Implementation_in_Pure_Java_Using_WSOLA 55. Timestretch Audio and Video Using 'Rubberband' Filter,
https://ffmpegbyexample.com/examples/749f6u35/timestretch_audio_and_video_using_rubberband_filter/ 56. rubberband - Audio time-stretching and pitch-shifting library and,
https://www.freshports.org/audio/rubberband/ 57. Free, Open-Source Audio Time-Stretching and Pitch-Shifting, https://superpowered.com/free-open-source-time-stretching-pitch-shifting 58. What is Wav2Lip? The open-source lip sync model | sync. labs blog,
https://sync.so/blog/what-is-wav2lip 59. LatentSync: Audio Conditioned Latent Diffusion Models for Lip Sync, https://arxiv.org/html/2412.09262v1 60. Best AI Lip Sync Generators (Open-Source / Free) in 2024,
https://www.pragnakalp.com/best-ai-lip-sync-generators-open-source-free-in-2024-a-comprehensive-guide/ 61. MuseTalk: Real-Time High Quality Lip Synchronization with Latent,
https://arxiv.org/html/2410.10122v2 62. A Lip Sync Expert Is All You Need for Speech to Lip Generation In,
https://purehost.bath.ac.uk/ws/files/211225619/Wav2Lip_ACMMM_20_camera_ready_2_.pdf 63. numz/wav2lip_studio - Hugging Face, https://huggingface.co/numz/wav2lip_studio 64. GitHub - ajay-sainy/Wav2Lip-GFPGAN: High quality Lip sync,
https://github.com/ajay-sainy/Wav2Lip-GFPGAN 65. Latent Sync: Advanced End-to-End Lip Synchronization Free, https://www.latentsync.org/ 66. Enhancing Lip Sync Quality: A Journey Through AI and Machine,
https://blog.makerx.com.au/enhancing-lip-sync-quality-a-journey-through-ai-and-machine-learning-models/ 67. Wav2Lip-CodeFormer - langzizhixin - GitHub,

https://github.com/langzizhixin/Wav2Lip-CodeFormer 68. README.md · numz/wav2lip_studio at, https://huggingface.co/numz/wav2lip_studio/blame/0ec441162089f96d1f4fae3a292f35438aaf7dcd/README.md 69. LSE-D: Lip-Sync Error-Distance Metric - Emergent Mind, https://www.emergentmind.com/topics/lse-d-lip-sync-error-distance 70. Audio-driven Talking Face Generation with Stabilized ... - ECVA, https://www.ecva.net/papers/eccv_2024/papers_ECCV/papers/02950-supp.pdf 71. MOVA: Towards Scalable and Synchronized Video–Audio Generation, https://arxiv.org/html/2602.08794v2 72. Understanding the Netflix Loudness Level in Editing Software, https://www.knuterikevensen.com/2022/06/04/measuring-the-netflix-loudness-level/ 73. Biometric Data GDPR Compliance Made Simple, https://gdprlocal.com/biometric-data-gdpr-compliance-made-simple/ 74. Biometrics - European Data Protection Board, https://www.edpb.europa.eu/topics/ai-and-technology/biometrics_en 75. What does the EU AI Act Article 50 mean for my Synthesia videos?, https://help.synthesia.io/en/articles/16046624-what-does-the-eu-ai-act-article-50-mean-for-my-synthesia-videos 76. Article 50 AI Act: Marking Outputs with C2PA & SynthID | Lyceum, https://lyceum.technology/magazine/article-50-ai-act-marking-outputs-with-c2pa/ 77. The EU AI Act's audio disclosure rules started August 2. Here ... - Vois, https://vois.so/blog/eu-ai-act-voice-disclosure-creators 78. AI Content Provenance and Watermarking: The PM's Guide to C2PA, https://www.institutepm.com/knowledge-hub/ai-content-provenance-watermarking 79. Python C2PA Tutorial: A Hands-on Guide to Verifying Images and, https://sightengine.com/python-c2pa-tutorial-verifying-images-detecting-tampering 80. c2pa-python · PyPI, https://pypi.org/project/c2pa-python/0.2.1/ 81. c2pa-python/docs/working-stores.md at main - GitHub, https://github.com/contentauth/c2pa-python/blob/main/docs/working-stores.md 82. Create C2PA Injected Files Easily with Numbers Protocol - Medium, https://medium.com/numbers-protocol/create-c2pa-injected-files-easily-with-numbers-protocol-56cc1f05bce4 83. A New Implementation Guide for Content Credentials - C2PA, https://c2pa.org/a-new-implementation-guide-for-content-credentials/ 84. C2PA Quick Start Guide, https://c2pa.wiki/getting-started/quick-start/ 85. C2PA Python library | Open-source tools for content authenticity and, https://opensource.contentauthenticity.org/docs/c2pa-python/